

Serious Games: Projektseminar

P01: ExerCubePainter

(Projekt-)Praktikum in the department of Electrical Engineering and Information Technology
by Julian Ewald, Atabey Tarik Akkum, Muammer Karayol, Katrin Pöppelmann
Date of submission: September 18, 2024

Reviewer: Dr.-Ing. (habil.) Stefan Göbel
Supervisor: Florian Horn, Polona Caserman
Darmstadt



TECHNISCHE
UNIVERSITÄT
DARMSTADT

Electrical Engineering and
Information Technology
Department
Serious Games

Erklärung zur Abschlussarbeit gemäß § 22 Abs. 7 APB TU Darmstadt

Hiermit erkläre ich, Julian Ewald, Atabey Tarık Akkum, Muammer Karayol, Katrin Pöppelmann, dass ich die vorliegende Arbeit gemäß § 22 Abs. 7 APB der TU Darmstadt selbstständig, ohne Hilfe Dritter und nur mit den angegebenen Quellen und Hilfsmitteln angefertigt habe. Ich habe mit Ausnahme der zitierten Literatur und anderer in der Arbeit genannter Quellen keine fremden Hilfsmittel benutzt. Die von mir bei der Anfertigung dieser wissenschaftlichen Arbeit wörtlich oder inhaltlich benutzte Literatur und alle anderen Quellen habe ich im Text deutlich gekennzeichnet und gesondert aufgeführt. Dies gilt auch für Quellen oder Hilfsmittel aus dem Internet.

Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

Mir ist bekannt, dass im Falle eines Plagiats (§ 38 Abs. 2 APB) ein Täuschungsversuch vorliegt, der dazu führt, dass die Arbeit mit 5,0 bewertet und damit ein Prüfungsversuch verbraucht wird. Abschlussarbeiten dürfen nur einmal wiederholt werden.

Bei einer Thesis des Fachbereichs Architektur entspricht die eingereichte elektronische Fassung dem vorgestellten Modell und den vorgelegten Plänen.

Darmstadt, 18. September 2024

J. Ewald, A. Akkum, M. Karayol, K. Pöppelmann

Contents

1	Projektdefinition	4
1.1	Projektidee	4
1.2	Zielgruppe	4
1.3	Herausforderungen	4
1.4	Verwendete Technologien	5
1.5	Ergebnis	5
2	Komponenten & Priorisierung	6
2.1	Zeichenfunktionalitäten	6
2.2	GUI	7
2.3	Tracking Logik	8
2.4	Metafunktionalitäten	8
2.5	KI Logik	9
3	Projektplanung	11
3.1	Projektplan	11
3.2	Gantt-Diagramm	12
3.3	Arbeitspakete	13
3.3.1	Zeichenfunktionalitäten	13
3.3.2	GUI	13
3.3.3	Tracking Logik	14
3.3.4	Metafunktionalitäten	14
3.3.5	KI Logik	15
4	Prototypische Entwicklung	16
4.1	Arbeitspakete	16
4.1.1	Zeichenfunktionalitäten	16
4.1.2	GUI	17
4.1.3	Tracking Logik	18
4.1.4	Metafunktionalitäten	18
4.1.5	KI-Logik	19
4.2	Evaluation	20
5	Zusammenfassung und Ausblick	21
5.1	Zusammenfassung	21
5.2	Lessons Learned	21
5.3	Ausblick	22
6	Anhang	23

1 Projektdefinition

1.1 Projektidee

Im Rahmen des Projekts soll ein Programm entwickelt werden, das es ermöglicht, innerhalb des ExerCube zu malen. Dabei wird das grob gezeichnete Bild durch den Einsatz künstlicher Intelligenz (KI) in eine ästhetisch ansprechendere Darstellung umgewandelt. Es soll eine kooperative Malfunktion implementiert werden, die es bis zu vier Personen gleichzeitig ermöglicht, gemeinsam an einem Kunstwerk zu arbeiten. Zusätzlich wird die Möglichkeit in Betracht gezogen, einen Wettbewerbsmodus für zwei Spieler zu entwickeln, wobei hierfür noch eine konkrete Konzeption fehlt.

1.2 Zielgruppe

Die Zielgruppe umfasst Menschen jeden Alters, die Freude an kreativer und künstlerischer Betätigung haben. Ein besonderes Interesse an der Erzeugung von Bildern mithilfe künstlicher Intelligenz ist von Vorteil. Auch professionelle Künstler und Grafikdesigner könnten von der Anwendung profitieren, indem sie den ExerCube für immersive Brainstorming-Sitzungen nutzen. Insbesondere dann, wenn mehrere Personen in den Entwurfsprozess eingebunden sind, könnte der ExerCube die Zusammenarbeit fördern.

1.3 Herausforderungen

Bei der Entwicklung des Projekts müssen mehrere technische und konzeptionelle Herausforderungen berücksichtigt werden:

- Eingabepräzision: Die Genauigkeit der Bewegungserfassung muss hoch genug sein, um eine angenehme Benutzerinteraktion zu gewährleisten.
- Reaktionszeit: Die Verzögerung zwischen Eingabe und Reaktion der Anwendung sollte minimal sein, um eine reibungslose Nutzererfahrung zu ermöglichen.
- Bildgenerierung: Die Bildgenerierung durch die KI muss effizient und in einer für die Nutzer angemessenen Zeit erfolgen.
- Intuitives Design: Die Benutzeroberfläche und Steuerung müssen einfach und intuitiv gestaltet sein, um eine breite Akzeptanz zu fördern.

1.4 Verwendete Technologien

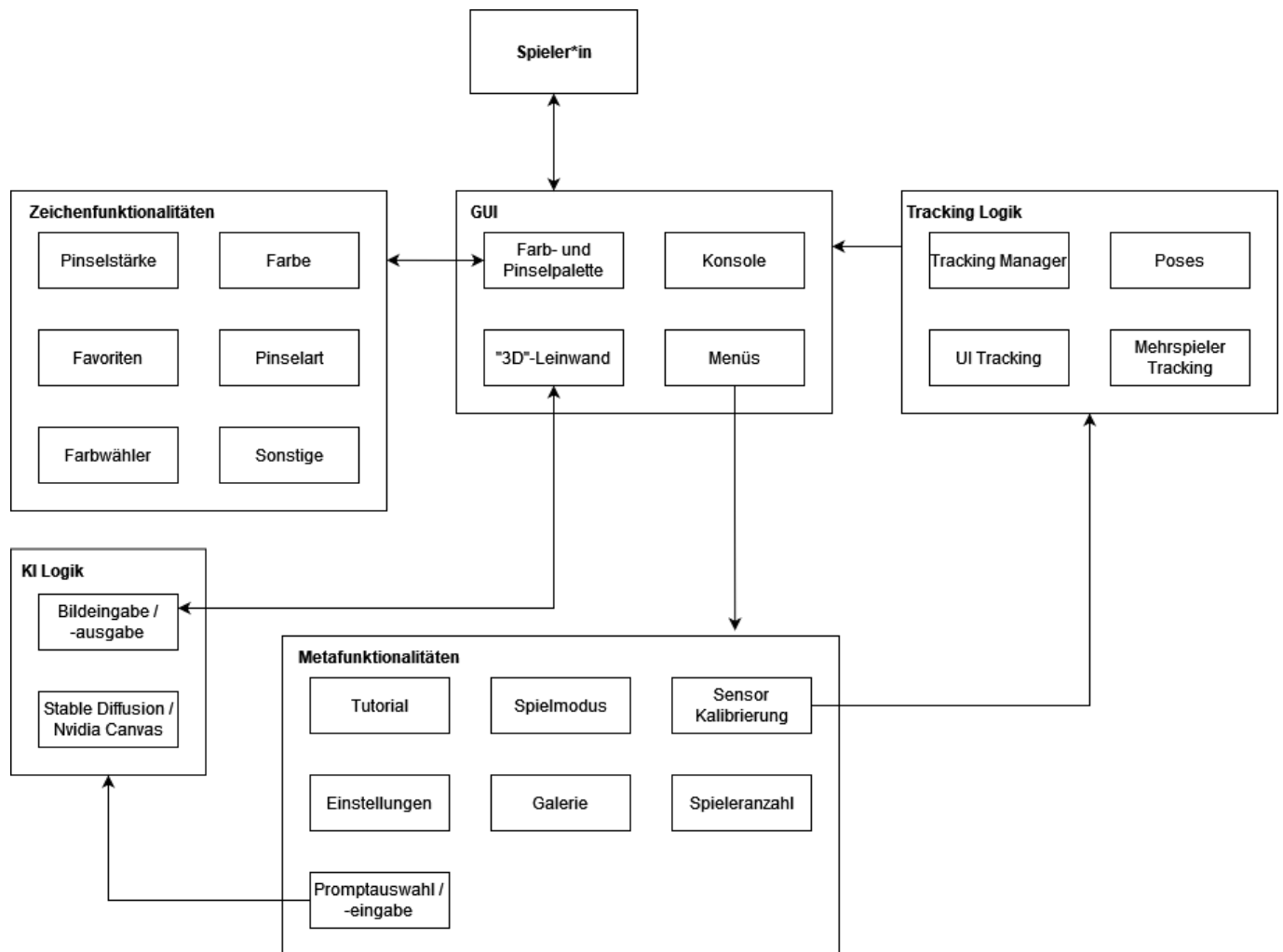
Für die Umsetzung des Projekts ist es geplant folgende Technologien zu verwenden, wobei für die KI zwei Auswahlmöglichkeiten in Betracht gewogen werden:

- Unity: Eine Game-Engine zur Implementierung der Anwendung und zur Erstellung der 3D-Umgebung.
- ExerCube: Ein immersives Trainings- und Spielsystem, das als Plattform für das Projekt dient.
- Stable Diffusion: Eine KI-Plattform für die Generierung von Bildern.
- NVIDIA Canvas: Ein Tool zur KI-unterstützten Bildgenerierung, das insbesondere für die Erstellung von Landschaften genutzt werden kann.

1.5 Ergebnis

Das Ziel des Projekts ist die Entwicklung einer spielerischen Zeichenanwendung für den ExerCube mit integrierter KI-Unterstützung. Der Grad der KI-Unterstützung soll variabel gestaltet werden, sodass der Nutzer zwischen geringerer Unterstützung – bei der die eigenen Ideen und künstlerischen Fähigkeiten im Vordergrund stehen – und einer stärkeren Unterstützung wählen kann, die auch unerfahrenen Nutzern ermöglicht, in kurzer Zeit beeindruckende Bilder zu erstellen. Die Steuerung der Anwendung erfolgt durch Bewegung innerhalb des ExerCube.

2 Komponenten & Priorisierung



2.1 Zeichenfunktionalitäten

Gesamte Personenstunden: 18h

Zeichnungsfunktionalitäten sind die Funktionalitäten im Zeichnungsteil des Spiels. Hier kann der Spieler mittels des Trackings und der ihm zur Verfügung stehenden Optionen auf Leinwänden zeichnen. Die hier gezeichneten Bilder werden dann über eine API an die künstliche Intelligenz weitergegeben, die diese Bilder als Input nimmt und daraus Bilder generiert. Must-have:

-
- Pinselstärke(4h): Für die Einstellung der Dicke des Pinselstrichs.
 - Pinselart(4h): Es gibt verschiedene Arten von Pinseln für unterschiedliche Zeichenstile.

Nice-to-have:

- Favoriten(2h): Hier können Benutzer ihre bevorzugten Pinsel, Farben und Optionen speichern und schnell darauf zugreifen.
- Farbwähler(4h): Ermöglicht es Benutzern, die Farbe des Zeichenwerkzeugs auszuwählen.
- Sonstige(4h): Dies umfasst weitere Funktionen oder Anpassungsmöglichkeiten, die für die Zeichenfunktionalität relevant sind.

Risikoanalyse:

- Es sollte darauf geachtet werden, dass die Übergänge zwischen den drei Bildschirmen beim Zeichnen korrekt sind.

2.2 GUI

Gesamte Personenstunden: 30h

Die GUI ist die Benutzeroberfläche des Spiels. Sie ist die zentrale Schnittstelle für den Spieler und die versteckte Spiellogik. Der Spieler nutzt die Schnittstelle um mit der Leinwand und anderen Funktionen zu interagieren und Tracking und KI Logik nutzen die GUI um die Spielereingaben oder die KI Ausgaben darzustellen: Must-have:

- Farb- und Pinselpalette(10h): Hier kann der Spieler die Farbe und die Art des Pinsels auswählen, mit dem er malen möchte.
- Konsole(4h): Die Konsole ist nur für Entwickler gedacht und kann verwendet werden, um die aktuelle Anwendung zu debuggen.
- "3D"-Leinwand(10h): Die Leinwand zeigt das aktuelle Bild an.
- Menüs(6h): Die Menüs bieten dem Spieler Zugriff auf verschiedene Funktionen des Spiels, wie Ton- und Bildeinstellungen.

Risikoanalyse:

- Um Barrierefreiheit zu gewährleisten, müssen farbenblindenfreundliche Farben für GUI Elemente gewählt werden.
- Darstellung und Anordnung der GUI Elemente müssen intuitiv sein, um Benutzerfreundlichkeit zu gewährleisten.
- Größe der Bedienfelder muss an Eingabegenauigkeit angepasst werden, um Fehleingaben zu vermeiden.
- Übergänge zwischen den drei Bildschirmen müssen bei der Anordnung der GUI Elemente beachtet werden, damit z.B. keine Bedienfelder oder Textanzeigen direkt am Übergang liegen und dadurch schwerer zugänglich sind.

2.3 Tracking Logik

Gesamte Personenstunden: 24h

Die Tracking Logik ist für die Verfolgung der Positionen der Spieler verantwortlich. Sie ist vorallem dafür verantwortlich die Bewegungseingaben des Spielers präzise auf UI Elemente und die Leinwand abzubilden. Sie hat eine Schnittstelle für den Spieler um die Tracker zu kalibrieren und hat eine Schnittstelle für die GUI um aktuelle Tracker Positionen darzustellen. Sie besteht aus folgenden Elementen: Must-have:

- Tracking Manager(6h): Der Tracking-Manager verwaltet alle Tracker im Spiel.
- Positionen(6h): Positionen beschreiben die aktuellen Positionen der Tracker.
- UI-Tracking(6h): Das UI-Tracking verfolgt die Interaktionen des Spielers mit den UI-Elementen.
- Mehrspieler-Tracking(6h): Das Mehrspieler-Tracking verwaltet die Eingaben mehrerer Spieler, um Mehrspieler Spielmodi zu ermöglichen.

Nice-to-have:

- Mehrspieler-Tracking(6h): Das Mehrspieler-Tracking verwaltet die Eingaben mehrerer Spieler, um Mehrspieler Spielmodi zu ermöglichen.

Risikoanalyse:

- Eingaben müssen zuverlässig auf Leinwand und Bedienfelder abgebildet werden.
- Tracker müssen zuverlässig funktionieren (Verbindungsverlust, Hardwareschwächen).
- Beim Mehrspieler Tracking müssen Eingaben der verschiedenen Tracker eindeutig einem Spieler zugeordnet werden und die Eingaben der Spieler dürfen sich nicht gegenseitig beeinflussen.

2.4 Metafunktionalitäten

Gesamte Personenstunden: 42h

Die Metafunktionalitäten sollen über die (in der GUI angezeigten) Menüs erreichbar sein. Darüber sollen Einstellungen getätigt werden, die vom normalen Spielfluss abweichen. Folgende Komponenten sind darin enthalten: Must-have:

- Einstellungen (4h): Hierunter sind mögliche Einstellungen zusammengefasst, die relevant sein könnten.
- Sensor Kalibrierung (8h): Es ist notwendig die Tracker rekalisieren zu können, falls die Position nicht richtig erkannt wird. Diese Auswahlmöglichkeit aktiviert dann eine Funktionalität in der Komponente Tracking Logik, um die Kalibrierung auszuführen.
- Spieleranzahl (*must-have*): Es sollen sich bis zu vier Spieler gleichzeitig registrieren können. Über diese Option soll man Spieler zuschalten und wegschalten können.

-
- Promptauswahl/-eingabe (8h): Diese Auswahl bildet eine Schnittstelle zur KI Logik. Nach Fertigstellung des Bildes soll es mit einem Prompt an das Stable Diffusion Modell / Nvidia Canvas gegeben werden. So kann beispielsweise der Stil oder die Beleuchtung des generierten Bildes modifiziert werden. Es gibt dazu vorgefertigte Eingaben die man nur auswählen muss (*must-have*) und falls machbar lässt sich auch eine individuelle textuelle Eingabe per Stift auf projizierter Tastatur tätigen (*nice-to-have*).

Nice-to-have:

- Tutorial (4h): Wenn der Spieler sich in der Anwendung noch nicht zurechtfindet, kann er hiermit das Tutorial starten, das die verschiedenen Funktionalitäten erläutert. Risiko: Die Funktionalitäten müssen erst weitgehend implementiert sein, bevor ein Tutorial erstellt werden kann.
- Spielmodus (10h): Hierüber soll sich der Spielmodus ändern lassen. Die Ideen sind ein normaler Zeichenmodus und ein spielerischer Modus, wobei das Spiel Gartic Phone als Inspiration dienen könnte.
- Galerie (8h): Hier soll der Spieler gespeicherte Bilder einsehen können.

Risikoanalyse:

- Textuelle Eingabe: Gedacht ist es, eine Tastatur an die Leinwand zu projizieren, die man mit dem virtuellen Stift bedienen kann. Falls die textuelle Eingabe per Stift schlecht funktioniert, könnte das Gartic Phone Spiel, das auf Texteingabe angewiesen ist, schwer zu implementieren sein, weshalb dieser Spielmodus weggelassen würde. Auch die individuelle Prompteingabe hängt davon ab.
- Mehrspielermodus: Es könnte schwierig werden, mehrere Spieler gleichzeitig Malen zu lassen, da dadurch mehrere Eingaben gleichzeitig verarbeitet werden müssen. Möglicherweise könnte es das System überlasten.

2.5 KI Logik

Gesamte Personenstunden: 37h

Ein wesentlicher Teil des Programms ist eine API für Nvidia Canvas / Stable Diffusion. Diese API hat zwei Eingaben (eine Prompt und ein gezeichnetes Bild) und liefert als Ausgabe ein Bild. Die KI interpretiert die Prompt zusammen mit der vom Benutzer erstellten Skizze und liefert ein Bild zurück. Must-have:

- Nvidia Canvas / Stable Diffusion (10h): Die Verwendung von Nvidia Canvas oder dem Stable Diffusion-Checkpoint 1.5 mit Scribble-ControlNet ist erforderlich.
- Python Script (20h): Die Parameter und Funktionen von Stable Diffusion müssen als Python Script parameterisiert werden.

Nice-to-have:

- Scribble-ControlNet(5h): Die Implementierung von Stable Diffusion-XL mit Scribble-ControlNet wäre wünschenswert.
- Skalierung(2h): Die Möglichkeit, das Bild zu skalieren.

Risikoanalyse:

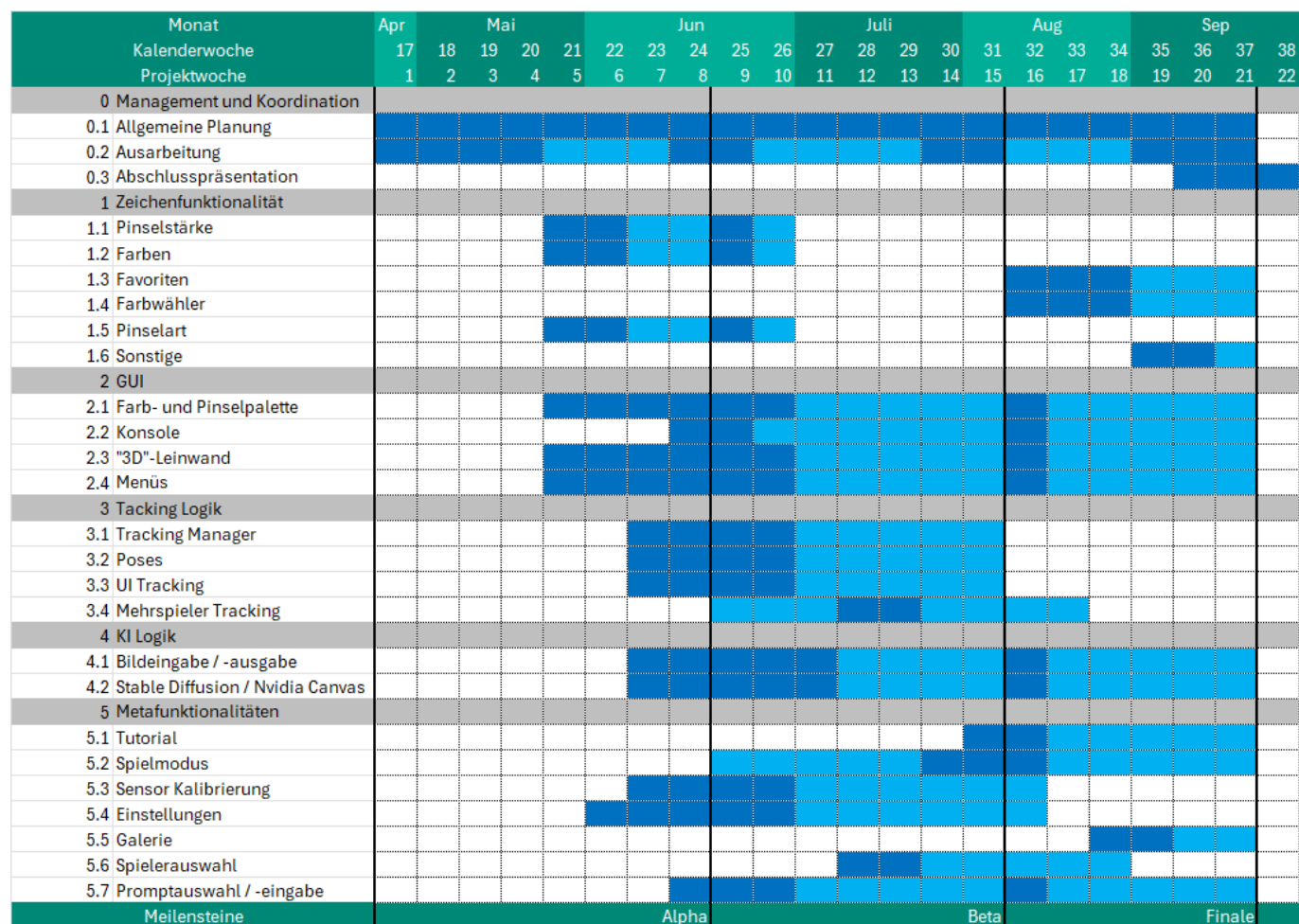
-
- Die Ergebnisse können aufgrund der Natur des künstlichen Intelligenzmodells zufällig sein. Als Lösung werden mehrere Bilder pro Zeichnung generiert.
 - Jeder API-Aufruf muss auf die Beendigung anderer API-Ausführungen warten. VRAM würde schnell gefüllt werden, wenn gleichzeitig 3 verschiedene Bilder angefordert würden.
 - Das Zielsystem verwendet Nvidia RTX 3060, was aufgrund seines 12GB Speichers möglicherweise eine Einschränkung darstellt. Als Lösung könnte Xformer und die Verwendung von lowvram-Argumenten funktionieren.
 - Bei Verwendung von Stable Diffusion mit einer ihrer Benutzeroberfläche besteht das Risiko einer Hardwareüberlastung.
 - Die Verwendung eines besseren Checkpoints würde die Bildstabilität und -qualität verbessern, erfordert jedoch möglicherweise zusätzliche Hardware. Es muss getestet werden, ob dies überhaupt möglich ist, da Scribble-ControlNet möglicherweise keine Unterstützung für Stable Diffusion XL bietet.

3 Projektplanung

3.1 Projektplan

Arbeitspakete (APs)	Lead	Personenarbeitstage (PAT)				Summe
		Julian	Atabey	Muammer	Katrin	
0 Management und Koordination	Julian	11	8	8	8	35
0.1 Allgemeine Planung		6	5	5	5	21
0.2 Ausarbeitung		2	2	2	2	8
0.3 Abschlusspräsentation		3	1	1	1	6
1 Zeichenfunktionalität	Katrin	~	~	~	15	15
1.1 Pinselstärke		~	~	~	2,5	2,5
1.2 Farben		~	~	~	2,5	2,5
1.3 Favoriten		~	~	~	1	1
1.4 Farbwähler		~	~	~	2,5	2,5
1.5 Pinselart		~	~	~	4	4
1.6 Sonstige		~	~	~	2,5	2,5
2 GUI	Julian	15	~	~	5	20
2.1 Farb- und Pinselpalette		6	~	~	2	8
2.2 Konsole		1	~	~	~	1
2.3 "3D"-Leinwand		1	~	~	~	1
2.4 Menüs		7	~	~	3	10
3 Tacking Logik	Atabey	2	2	6	~	10
3.1 Tracking Manager		~	1	1	~	2
3.2 Poses		~	~	2	~	2
3.3 UI Tracking		~	1	1	~	2
3.4 Mehrspieler Tracking		2	~	2	~	4
4 KI Logik	Atabey	~	18	7	~	25
4.1 Bildeingabe / -ausgabe		~	10	5	~	15
4.2 Stable Diffusion / Nvidia Canvas		~	8	2	~	10
5 Metafunktionalitäten	Muammer	17	2	9	2	30
5.1 Tutorial		5	~	~	~	5
5.2 Spielmodus		4	2	2	2	10
5.3 Sensor Kalibrierung		1	~	2	~	3
5.4 Einstellungen		3	~	~	~	3
5.5 Galerie		3	~	~	~	3
5.6 Spielerauswahl		1	~	~	~	1
5.7 Promptauswahl / -eingabe			~	5	~	5
Gesamtaufwand		45	30	30	30	135

3.2 Gantt-Diagramm



Jeder Leiter eines Arbeitspaketes hat folgende Hauptaufgaben:

1. Risikomanagement: Identifizierung potenzieller Risiken und Entwicklung von Strategien zur Risikominimierung.
2. Fortschrittskontrolle: Regelmäßige Überprüfung des Fortschritts im Vergleich zum Zeitplan und Ergreifen von Korrekturmaßnahmen bei Abweichungen.
3. Qualitätssicherung: Sicherstellung, dass die Ergebnisse des Arbeitspakets den Qualitätsstandards entsprechen.

3.3 Arbeitspakete

3.3.1 Zeichenfunktionalitäten

Lead: Katrin

Definition of Done:

- Pinselstärke: Es lassen sich verschiedene Pinselstärken verwenden. Die Pinselstärke korrespondiert mit einem Zahlenwert, der sich in der GUI durch einen Slider oder etwas ähnlichem eingeben lässt. Dafür wurden 2,5 PAT eingeplant.
- Farben: Die Farbe des Pinsels lässt sich einstellen. Möglich wäre eine kreisförmige Auswahlpalette oder auch vorgefertigte Farbwerte. Dafür wurden 2,5 PAT eingeplant.
- Favoriten: Bestimmte Farben oder Pinselarten lassen sich als Favoriten in eine Kurzwahl eintragen, um eine nutzerfreundliche Bedienung zu gewährleisten. Dafür wurde 1 PAT eingeplant.
- Farbwähler: Ein Farbwähler lässt sich auf der Leinwand anwenden, um die Farbe des gewählten Pixels auszusuchen. Dafür wurden 2,5 PAT eingeplant.
- Pinselart: Verschiedene Pinselarten wie zum Beispiel Füller, Bleistift oder Sprühfarben-Look sollen angeboten werden. Dafür wurden 4 PAT eingeplant.
- Sonstige: Möglicherweise stellt sich im Laufe der Entwicklung heraus, dass noch Features fehlen, die die User Experience verbessern würden. Für diese Eventualität wurden 2,5 PAT eingeplant.

3.3.2 GUI

Lead: Julian

Definition of Done:

- Farb- und Pinselpalette: Eine grafische Darstellung der möglichen Auswahlmöglichkeiten, welche intuitiv und benutzerfreundlich gestaltet ist. Dafür wurden 8 PAT eingeplant.
- Konsole: Ausgabe von Meldungen und Informationen (Fehler, Status, etc.) in Echtzeit und Möglichkeit zur Eingabe von Befehlen durch den Benutzer. Dafür wurde 1 PAT eingeplant.
- "3D"-Leinwand: Die Anwendung von einem 2D-Bildschirm auf den 3D-ExerCube übertragen. Dafür wurde 1 PAT eingeplant.
- Menüs: Alle Menüpunkte sind implementiert, funktionsfähig und reagieren korrekt auf Benutzereingaben. Dafür wurden 10 PAT eingeplant.

3.3.3 Tracking Logik

Lead: Atabey

Definition of Done:

- Tracking Manager: Nimmt die Position jedes Trackers entgegen und gibt den Cursor auf dem entsprechenden Bildschirm aus. Dafür wurden 2 PAT eingeplant.
- Poses: Liefert die Positionen der Tracker in Echtzeit. Dafür wurden 2 PAT eingeplant.
- UI Tracking: UI-Elemente wissen, ob sich der Cursor über ihnen befindet, mit Hilfe des Tracking-Managers, und reagieren entsprechend. Dafür wurden 2 PAT eingeplant.
- Mehrspieler Tracking: Mindestens 3 Tracker können gleichzeitig verwendet werden. Dafür wurden 4 PAT eingeplant.

3.3.4 Metafunktionalitäten

Lead: Muammer

Definition of Done:

- Tutorial: Es sollte ein Tutorial geben, auf das der Benutzer zugreifen kann, um zu lernen, wie man die Anwendung verwendet. Dafür wurden 5 PAT eingeplant.
- Spielmodus: Die Anwendung sollte einen oder mehrere Spielmodi haben, um den Spaßfaktor zu erhöhen. Da noch nicht klar ist, wie die Spielmodi aussehen werden, sollten sie geplant und dann entsprechend implementiert werden. Dafür wurden 10 PAT eingeplant.
- Sensor Kalibrierung: Es sollte möglich sein, das Tracking in der Anwendung neu zu kalibrieren, wenn es nicht wie gewünscht funktioniert. Dafür wurden 3 PAT eingeplant.
- Einstellungen: Ein Sektion für Einstellungen sollte implementiert werden, um verschiedene Teile der Anwendung anzupassen. Dazu wurden 3 PAT eingeplant.
- Galerie: Es sollte eine Galerie geben, in der der Benutzer die gespeicherten Bilder ansehen kann. Dafür wurde 3 PAT eingeplant.
- Spielerauswahl: Die Anzahl der Benutzer für den Spielmodus sollte wählbar sein. Dafür wurde 1 PAT eingeplant.
- Promptauswahl/-eingabe: Es sollte die Möglichkeit geben, bei der Übertragung der Zeichnung des Benutzers an die KI Optionen aus einer Liste auszuwählen oder in einem Textfeld frei zu schreiben. Diese sollten dann als Prompts an die KI weitergegeben werden. Dafür wurden 5 PAT eingeplant.

3.3.5 KI Logik

Lead: Atabey

Definition of Done:

- Bildeingabe/-ausgabe: Ein Python-Skript, das als API für Stable Diffusion fungiert, ist geplant. Es sollte möglich sein, jeden Eingabewert von Stable Diffusion zu ändern. Dafür wurden 15 PAT eingeplant.
- Stable Diffusion/Nvidia Canvas: Stable Diffusion oder Nvidia Canvas kann Zeichnungen und Eingaben entgegennehmen und dann ein KI-generiertes Bild ausgeben. Die Bilder müssen den gegebenen Zeichnungen entsprechen. Dafür wurden 10 PAT eingeplant.

4 Prototypische Entwicklung

Im folgenden Kapitel wird die Herangehensweise an jedes einzelne Arbeitspaket beschrieben und der finale Stand des Prototypen festgehalten. Anschließend erfolgt eine Evaluation des Prototypen.

4.1 Arbeitspakete

4.1.1 Zeichenfunktionalitäten

Herangehensweise: Im Rahmen der Projektplanung wurden verschiedene Ideen zur Implementierung der Zeichenfunktionalitäten gesammelt. Im Laufe des Projekts stellte sich jedoch heraus, dass ein minimalistisches Design nutzerfreundlicher und übersichtlicher ist. Es wurde eine klare Auswahl an 12 Farben und vier Pinselgrößen implementiert. Darüber hinaus wurde die Option hinzugefügt, die Leinwand zurückzusetzen, obwohl dies ursprünglich nicht in der Planung berücksichtigt war. Eine Auswahl von Favoriten wurde nicht umgesetzt, da die Optionen als übersichtlich genug erachtet wurden. Auch unterschiedliche Pinselarten wurden nicht implementiert, jedoch wäre dies eine mögliche Erweiterung für zukünftige Entwicklungen. Ein Farbwähler wurde als überflüssig eingestuft, da die Anzahl der Farben so gering ist, dass eine eindeutige Zuordnung durch die Nutzer problemlos möglich ist.

Die Farb- und Pinselgrößenauswahl wurde von der Plattform "Skribbl.io" inspiriert. Diese Farbpalette enthält genügend Farben, um die meisten Begriffe problemlos darstellen zu können. Auch bei Tests erwies sich die Auswahl als ausreichend.

Technische Dokumentation: Die Zeichenfunktionalitäten werden mittels eines "DrawingScripts" umgesetzt, welches sich in "Assets/Exercube/Painter/Scripts" befindet. Dieses Script ist Komponente eines RawImage (der Leinwand) und enthält die aktuellen Parameter für Farbe und Pinselgröße sowie die Verwaltung der Indikatorposition. Zudem wird in diesem Script überprüft, ob und welches UI-Element gerade ausgewählt ist.

Zusätzliche Funktionalitäten können leicht durch UI-Buttons ergänzt werden, da dynamisch überprüft wird, welcher Knopf gedrückt wurde, um dann eine vordefinierte Funktion auszuführen. Mögliche Erweiterungen wären hierbei eine Undo-Funktion (zum Zurücksetzen des letzten Pinselstrichs) oder die Implementierung verschiedener Pinselarten.

Im Verlauf der Implementierung wurden keine neuen Abhängigkeiten eingeführt, abgesehen von den bereits im README des GitHub-Repositories genannten.

4.1.2 GUI

Herangehensweise: Für die Entwicklung der Benutzeroberfläche (GUI) wurden zunächst erfolgreiche Multiplayer-Spiele wie "Scribbl.io" analysiert, um Inspiration zu gewinnen. Anschließend wurde ein Prototyp entworfen und im Exercube getestet sowie an die Anforderungen angepasst.

Mithilfe von VR-Trackern sollte es den Spielern ermöglicht werden, intuitiv und direkt mit den Elementen auf dem Bildschirm zu interagieren. Um eine präzise Steuerung zu gewährleisten, wurde die Größe der Bedienelemente an die Präzision der Tracker angepasst.

Da die Bewegungssteuerung eine gewisse Ungenauigkeit mit sich bringt, wurden zur Vermeidung von unbeabsichtigten Aktionen sogenannte Slider eingeführt. Diese visuell dargestellten Timer geben dem Spieler eine klare Rückmeldung darüber, wann eine Auswahl bestätigt wird. Beispielsweise wurden für die Farb- und Pinselauswahl kreisförmige Indikatoren verwendet, die sich füllen während der Tracker auf dem entsprechenden Element verweilt. Der Indikator verändert dabei Größe und Farbe, um die aktuelle Auswahl anzuzeigen. Diese Maßnahmen tragen zu einer intuitiven und fehlertoleranten Benutzerführung bei. Zum besseren Verständnis ist ein Sliderelement in Abbildung 6.2 dargestellt.

Ein weiterer Aspekt bestand darin, eine Debugging-Umgebung sowie ein Menü für die Eingaben zur Kalibrierung der Tracker und für die KI bereitzustellen. Hierfür wurde das vorhandene Tablet verwendet, da die Texteingabe darauf komfortabler ist. Auf dem Tablet sind dauerhaft eine Konsole, Indikatoren für die verbundenen Tracker und ein dynamisches Menü sichtbar, dessen Schaltflächen je nach Spielmodus variieren.

Nachdem die grundlegenden UI-Elemente (Leinwand, Farbpalette, Pinselstärke; siehe Abschnitt 4.1.1) fertiggestellt waren, wurde das Design des Spielstarts und der Auswahl der Spielmodi umgesetzt, die ebenfalls über die Tracker erfolgen sollte. Hintergrund, Schriftart und Farben wurden so gewählt, dass eine spielerische und futuristische Atmosphäre entsteht.

Im weiteren Verlauf wurden fortlaufend kleinere Fehler behoben und die Benutzeroberfläche kontinuierlich verfeinert.

Technische Dokumentation: Für die GUI selbst wurde ein "ScrollerScript" eingeführt, welches sich in "Assets/Exercube/Painter/Scripts" befindet. Dieses Script lässt ein Hintergrundbild durch das Bild scrollen. Neu eingeführte Texturen befinden sich in "Assets/Exercube/Painter/Textures". Im allgemeinen wurde für jedes vorhandene Display bzw. Camera (Tablet, Front, Left, Right) ein Canvas in Unity erstellt. Auf diesen Canvases wurden dann die UI Elemente erstellt.

Die GUI ist recht schwer erweiterbar, da der ExerCube einige Schwierigkeiten beim UI-Designen einführt (Relation, abweichende Größe, abweichend Position). Im allgemeinen könnte man die GUI noch weiter ausbauen, damit Szenenübergänge und das allgemeine Aussehen "professioneller" sind.

Für die Entwicklung der GUI wurden ausschließlich Standardfunktionen von Unity verwendet. Grafiken, wie der Hintergrund der Spieleauswahl, wurden teilweise mit GIMP erstellt. Ansonsten kamen keine neuen Abhängigkeiten oder zusätzliche Technologien zum Einsatz.

4.1.3 Tracking Logik

Herangehensweise: Dieses Arbeitspaket konnte erst begonnen werden, nachdem die grundlegenden Zeichenfunktionen und die GUI implementiert waren.

Schrittweise wurde bei den unabhängig des Exercubes erstellten Prototypen die Eingabe per Computermaus durch die Eingabe mit den VR-Trackern ersetzt.

Für einen Großteil der Funktionalitäten wurde auf den vorhandenen Code des Exercube SDK zurückgegriffen, der dann entsprechend den Anforderungen angepasst wurde. Der im Rahmen des Projekts entwickelte Code konzentriert sich hauptsächlich darauf, die Präzision der Eingaben zu optimieren.

Der allgemeine Ansatz besteht darin, dass der im Abschnitt 4.1.2 erwähnte Tracker sowohl für die Position des Pinsels als auch für die Auswahl der UI-Elemente verantwortlich ist. Die Position des Trackers wird so angepasst, dass sie mit der aktuellen Position des Indikators übereinstimmt, wodurch das Problem der Präzision relativ einfach gelöst wird.

Das Mehrspieler-Tracking stellte sich als unkompliziert heraus, da zu Beginn des Spiels jedem Tracker eine eindeutige Spielernummer zugewiesen wird. Dadurch kann die Position jedes Trackers eindeutig einer der Leinwände zugeordnet werden.

Technische Dokumentation: Die Tracking-Logik war Teil des mitgelieferten ExerCube SDK, sodass wenig bis gar kein eigener Code geschrieben werden musste. Die Implementierungen befinden sich im Verzeichnis "Assets/Exercube/Tracking". Einige Funktionen des SDK wurden deaktiviert, da sie mit den eigenen Implementierungen interferierten (beispielsweise Hotkeys).

Die Tracking-Logik ist leicht erweiterbar und anpassbar, da standardmäßig bis zu acht Tracker unterstützt werden und die Positionsbestimmung der Tracker eine hohe Genauigkeit aufweist. Eine Erweiterung des Mappings von Trackern zu In-Game-Objekten gestaltet sich jedoch als komplexer.

Da die Tracking-Logik Teil des SDK ist, wurden keine zusätzlichen Abhängigkeiten eingeführt.

4.1.4 Metafunktionalitäten

Herangehensweise: Die Implementierung der Metafunktionalitäten konnte erst in einem späteren Stadium der Spieleentwicklung begonnen werden.

Ein Tutorial wurde in Form von kleinen Animationen umgesetzt, die die Elementauswahl zeigen. Diese Umsetzung wurde von Spielanleitungsbildschirmen, wie sie in Spielen wie Mario Party verwendet werden, inspiriert, wo ein kleines GIF in Dauerschleife die Mechaniken erklärt.

Die Auswahl von Spielmodi und der Spieleranzahl erfolgt durch den Einsatz von Trackern, um die Bewegungseingabe hervorzuheben. Verschiedene Spielmodi wurden in der Gruppe diskutiert, und eine Auswahl wurde implementiert. Zur Erkundung der KI-Funktionalitäten wurde ein "Expert-Modus" eingeführt, in dem frei gezeichnet und der Text-Prompt eigenständig angepasst werden kann. Zudem wurden zwei Fun-Modi entwickelt, in denen den Spielern Begriffe vorgegeben werden, die sie in begrenzter Zeit zeichnen müssen. Eine genauere Beschreibung der Spielmodi erfolgt in Abschnitt 5.1.

Die Auswahl des Prompts erfolgt nur im "Expert-Modus" über das Tablet. In den anderen Spielmodi wird der vorgegebene Begriff als Prompt verwendet. Eine Sektion für Einstellungen sowie eine Galerie im ExerCube wurden nicht umgesetzt.

Technische Dokumentation: Die Metafunktionalitäten werden durch das "StartScript" und "Gamemode-Script" gesteuert, welche sich im Verzeichnis "Assets/Exercube/Painter/Scripts" befinden. Diese Skripte sind jeweils Komponenten des "StartManager" oder "GamemodeManager" in den verschiedenen Szenen. Sie sind dafür verantwortlich, dass die entsprechenden UI-Elemente ein- und ausgeblendet, Spielmodi gestartet und Prompts für die KI geändert werden. Die Skripte enthalten somit alle Metafunktionalitäten. Abhängig von der aktuellen Szene wird der entsprechende Manager aktiviert. Ein zusätzliches "StaticData"-Skript wird verwendet, um den Datenaustausch zwischen den Skripten zu ermöglichen.

Für die Entwicklung der Metafunktionalitäten wurden ausschließlich Standardfunktionen von Unity verwendet, sodass keine neuen Abhängigkeiten oder zusätzlichen Technologien entstanden sind.

4.1.5 KI-Logik

Herangehensweise: Zu Beginn des Projekts wurden umfangreiche Recherchen durchgeführt, um geeignete KI-Tools für den Anwendungsfall zu identifizieren. Nach sorgfältiger Abwägung kamen nur zwei Optionen in Frage: Stable Diffusion und NVIDIA Canvas. Da NVIDIA Canvas jedoch ausschließlich die Generierung von Landschaften unterstützt und auch Tiere sowie Menschen dargestellt werden sollten, fiel die Entscheidung auf Stable Diffusion.

Die erste Implementierung basierte auf Stable Diffusion in Kombination mit Automatic1111, einer WebUI, die die Verwaltung von Stable Diffusion-Modellen erleichtert. Die Bildgenerierung erfolgte über API-Calls. Zunächst wurde das Modell stable-diffusion-xl-base-1.0 zusammen mit dem sd-controlnet-scribble-Modul eingesetzt, um die Erzeugung von Bildern auf Basis von Skizzen zu steuern.

Die anfänglichen Ergebnisse erwiesen sich als vielversprechend. Der generative Prozess funktionierte, indem die KI einen Text-Prompt erhielt, beispielsweise "red turtle", während das ControlNet die gemalten Skizzen zur Steuerung der Bildausgabe verwendete. Beispielbilder sind in Abbildung 6.1 dargestellt.

Das Hauptproblem stellte die lange Rechenzeit bei der Bildgenerierung dar. Da SDXL ein relativ großes Modell ist, dauerte der Prozess entsprechend lange. Durch ein Meeting mit "Studio Mux", einem Unternehmen, das digitale Exponate für Ausstellungen entwickelt, wurde auf "Stream Diffusion" aufmerksam gemacht. Dieser Austausch wurde auf Empfehlung des Betreuers initiiert.

Stream Diffusion basiert ebenfalls auf Stable Diffusion, arbeitet jedoch schneller und benötigt weniger Rechenleistung. Dadurch kann dem Nutzer ein nahezu unmittelbares Feedback darüber gegeben werden, wie die KI seine Eingaben interpretiert. Ein Beispielbild ist in Abbildung 6.3 zu sehen.

Stream Diffusion wird im Hintergrund auf einem Flask-Server gehostet, um eine effiziente und benutzerfreundliche Interaktion zu gewährleisten.

Technische Dokumentation: Die KI Logik wird mittels dem "StableDiffusionVideoToVideo" Script, welches sich in "Assets/Exercube/Painter/Scripts" befinden und dem Stream Diffusion Server, welcher sich in "SD_SERVER" befindet geregelt. Im "StableDiffusionVideoToVideo" Script werden die gezeichneten Bilder an den "SD_SERVER" geschickt und dann Antworten wieder verarbeitet und auf einem vorher festgelegten Canvas angezeigt.

Die Integration dieses KI-Backends brachte eine Vielzahl von Abhängigkeiten mit sich. Das KI-System wird in einer Python-Umgebung mittels eines Virtual Environment gehostet. Das Einrichten des Virtual Environment und aller weiteren Abhängigkeiten erfolgt über ein Installationsskript.

Zum Starten des KI-Backends muss lediglich das Startskript ausgeführt werden. Sobald das Skript läuft, öffnet sich ein Terminal, in dem die IP-Adresse sowie der Port der Stream Diffusion-Pipeline angezeigt werden. Der finale Code ermöglicht derzeit die simultane Erstellung von drei Stream Diffusion-Pipelines, sodass zukünftig bis zu drei Bilder gleichzeitig verarbeitet werden können.

Wenn der Server jedoch nicht läuft, funktioniert auch die Generierung der KI-Bilder nicht. Somit wurde hier eine neue Abhängigkeit eingeführt.

4.2 Evaluation

Zur Evaluierung des Fortschritts der einzelnen Arbeitspakete wurde die zuvor festgelegte "Definition of Done" herangezogen. Die Bewertung erfolgte in drei Testphasen, ergänzt durch kleinere Tests während der Entwicklung der jeweiligen Funktionen.

Die erste Testphase umfasste die Überprüfung der grundlegenden Funktionen. Dazu zählten die anfängliche GUI (3D-Leinwand, Auswahl der Pinselstärke, Farbauswahl, Pinselanzeige), die Tracking-Logik (Prüfung der Genauigkeit des Mappings von Tracker zu Indikator), die KI-Logik (Verarbeitung von Bilddaten bei Eingabe und Ausgabe) sowie die Zeichenfunktionalität (Überprüfung der Funktionalität der Auswahlmöglichkeiten in der GUI).

Das Design der GUI wurde in der Gruppe diskutiert und getestet, wobei insbesondere auf die Größe und Erreichbarkeit der einzelnen GUI-Elemente geachtet wurde. Die Tracking-Logik wurde evaluiert, indem der Tracker zu den äußeren Rändern der Leinwand bewegt und die Genauigkeit des Mappings überprüft wurde. Bei der KI-Logik lag der Fokus darauf, ob ein Output generiert wird und die Verarbeitungsgeschwindigkeit ausreichend ist (Generierung von Bildern alle 3 Sekunden). Zudem wurden verschiedene Hyperparameter als Eingaben getestet und zudem überprüft, welche Prompts die Qualität des generierten Bildes verbessern. Die Zeichenfunktionalität wurde durch das Anfertigen von Beispielbildern überprüft und anschließend in der Gruppe diskutiert.

In der zweiten Testphase standen die Metafunktionalitäten im Fokus. Dabei wurden das erstellte Tutorial, die verschiedenen Spielmodi, die Spielerauswahl sowie die Prompteingabe per Tablet getestet.

Da die grundlegenden Funktionen bereits in der ersten Testphase erfolgreich validiert wurden, gestaltete sich die zweite Testphase relativ unkompliziert. Die neu implementierten Features wurden von den Gruppenmitgliedern erprobt und im Anschluss diskutiert, um mögliche Verbesserungsmöglichkeiten zu identifizieren.

Die dritte Testphase diente der Feinabstimmung. Kleinere Fehler wurden behoben, und einige Aspekte wurden modifiziert, um das Spielerlebnis weiter zu optimieren. Eine detaillierte Auflistung aller Änderungen würde den Rahmen sprengen; das Spiel wurde iterativ getestet, wobei nach jeder Verbesserung ein weiterer Test durchgeführt wurde.

5 Zusammenfassung und Ausblick

5.1 Zusammenfassung

Das entwickelte Spiel umfasst zwei Hauptszenen und bietet den Spielern eine immersive und interaktive Erfahrung. In der ersten Szene wird ein kurzes Intro mit dem Spiel-Logo abgespielt, gefolgt von einem Tutorial, das die wesentlichen Funktionen des Spiels erklärt. Nach dem Tutorial gelangt der Spieler in die Auswahl des Spielmodus. Hier kann der gewünschte Modus durch eine Interaktion mit dem Tracker ausgewählt werden, indem dieser in die Nähe der Leinwand gebracht wird, auf der die verschiedenen Spielmodi angezeigt werden. Anschließend wird die Anzahl der Mitspieler auf ähnliche Weise gewählt.

In der zweiten Szene stehen die drei unterschiedlichen Spielmodi zur Verfügung:

- *Expert-Modus*: Dieser Modus ermöglicht es den Spielern, die KI ohne spezifische Vorgaben auszuprobieren. Auf der vorderen Leinwand können Zeichnungen erstellt werden, während die KI-generierten Bilder auf zwei weiteren Leinwänden angezeigt werden. Über das Tablet kann der Text-Prompt der KI angepasst werden, um unterschiedliche Bildausgaben zu erzeugen. Beispielsweise könnte der Stil des erzeugten Bildes dadurch justiert werden, um einen realistischen, abstrakten oder dramatischen Zeichenstil zu erhalten.
- *Pinzel-Panik-Modus*: In diesem Modus muss eine Person einen Begriff zeichnen, während die anderen Spieler diesen erraten. Der besondere Twist besteht darin, dass die ratenden Spieler nur die von der KI generierten Bilder sehen und auf Basis dieser das gezeichnete Objekt erraten müssen.
- *Canvas-Chaos-Modus*: Hier wird den Spielern ein Begriff vorgegeben, und sie müssen innerhalb eines begrenzten Zeitraums zusammenarbeiten, um ein Bild zu erstellen.

Das finale Produkt befindet sich im "release" branch auf github und ist erreichbar unter <https://github.com/serious-games-darmstadt/ExerCubePainter/tree/release>.

5.2 Lessons Learned

Im Laufe des Projekts wurde deutlich, dass die Zeitplanung und der Umfang der Arbeit nicht optimal aufeinander abgestimmt waren. Einige geplante Funktionalitäten konnten aufgrund des begrenzten Zeitrahmens nicht vollständig umgesetzt werden. Besonders die technische Implementierung auf dem Exercube stellte eine unerwartet große Herausforderung dar, was zu Verzögerungen in der Entwicklung führte. Das Debuggen der Anwendung war durch das manuelle Bauen und Kopieren auf den Rechner des Exercubes recht mühselig und erforderte häufiges Testen vor Ort. Diese Problematik wurde in der Zeitplanung nicht berücksichtigt.

5.3 Ausblick

Für zukünftige Arbeiten gibt es mehrere Ansatzpunkte zur Weiterentwicklung des Projekts. Ein Bereich mit Verbesserungspotenzial ist die Verfeinerung der Zeichenfunktionalitäten, beispielsweise durch verschiedene Pinselarten. Ein Button zum Rückgängig-Machen eines Pinselstrichs würde die Nutzerfreundlichkeit erhöhen. Um die Anwendung als ernstzunehmendes Zeichenprogramm weiterzuentwickeln wären viele derartige Verbesserungen notwendig, die den Rahmen des Projekts gesprengt hätten, jedoch auch für einen Nutzer interessant sein könnten, der hochwertige Bilder in dem Exercube malen möchte.

Zudem könnten weitere Spielmodi integriert werden, um die Vielfalt und den Wiederspielwert zu erhöhen. Auch die Implementierung von fortgeschrittenen KI-Algorithmen zur Bildgenerierung könnte ein spannendes Thema für weiterführende Arbeiten sein, denn da in diesem Feld laufend Fortschritte erzielt werden, ist eine rapide Verbesserung der Modelle zu erwarten.

6 Anhang

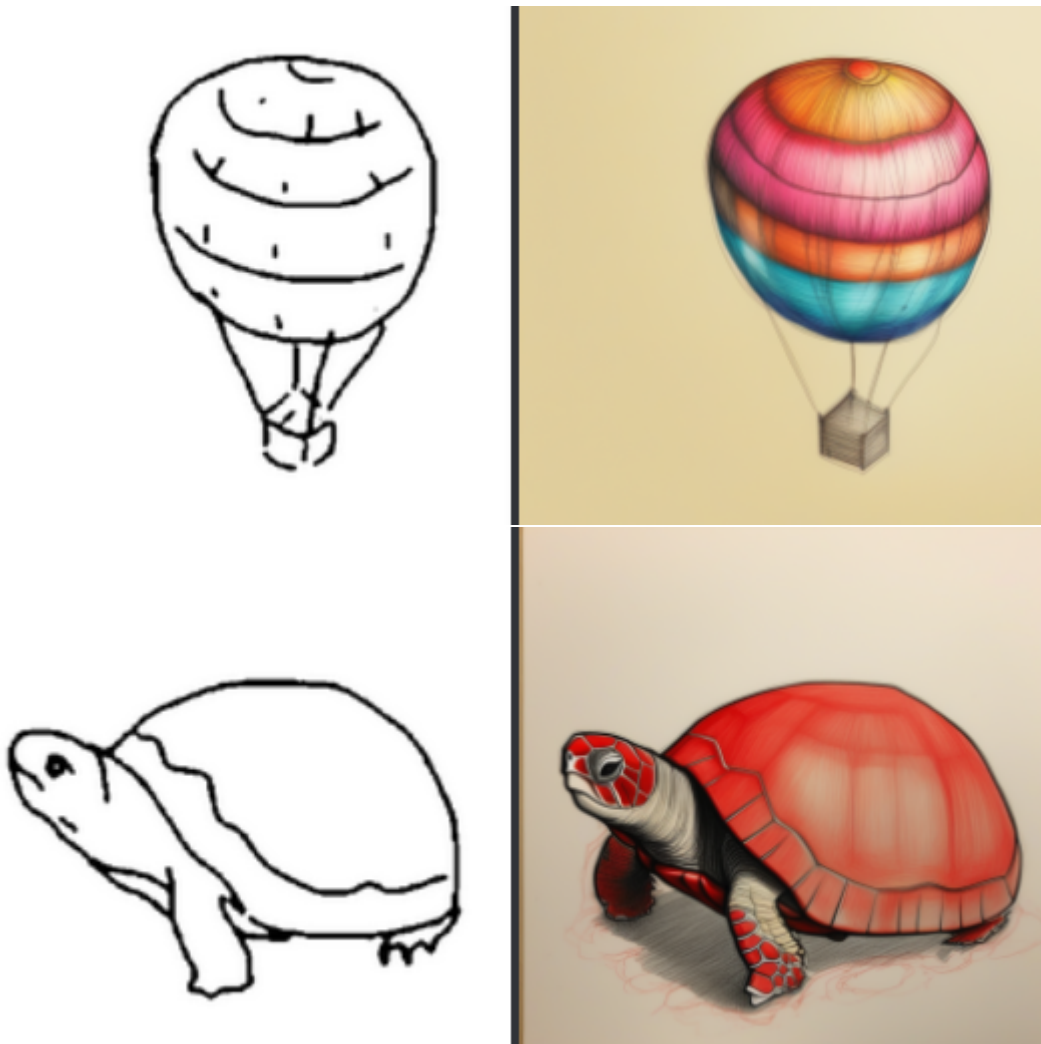


Figure 6.1: Beispielbilder, die mit ControlNet erzeugt wurden, siehe <https://github.com/lllyasviel/ControlNet>

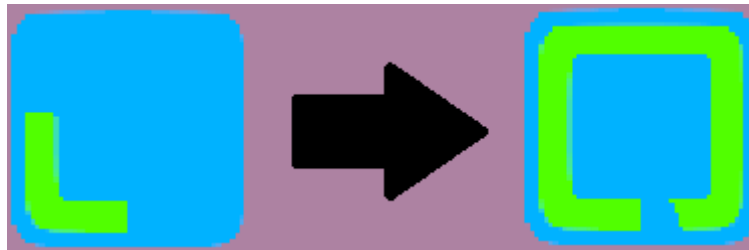


Figure 6.2: Illustrierung der Funktionsweise eines Sliders.



Figure 6.3: Beispielbild aus dem StreamDiffusion GitHub-Repository, siehe <https://github.com/cumulo-autumn/StreamDiffusion>